PROJECT DOCUMENTATION

STUDENT PERFORMANCE ANALYTICS SYSTEM

Submitted By
Arpita Laxman Adhude
Course: Python with AI
Technology: Python, NumPy, Pandas
Project Type: Python Data Analysis Project
Academic Year: 2026

Student Performance Analytics System

CERTIFICATE

This is to certify that Arpita Laxman Adhude has completed the mini project titled “Student Performance Analytics System” as part of the Python with AI course. The project demonstrates the practical application of Python programming fundamentals, NumPy, Pandas, functions, loops, conditional statements and data analysis techniques.

The project has been prepared as an individual academic assignment and is intended to demonstrate the student's understanding of handling structured student data and converting it into meaningful performance insights.

Date: _____

Trainer / Faculty Signature: _____

Institute / Course: EWB Courses – Python with AI

DECLARATION

I, Arpita Laxman Adhude , hereby declare that the project titled “Student Performance Analytics System” is prepared by me for academic and learning purposes as part of the Python with AI course.

I have used Python, NumPy and Pandas to load, process and analyze the student dataset. The project is intended to demonstrate concepts taught during the course, including variables, data types, loops, functions, conditions, DataFrames and numerical analysis.

I understand that the project should be tested before submission and that the documentation should accurately describe the implementation and results.

Place: _____

Date: _____

Signature: Arpita Laxman Adhude

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my trainer and faculty for providing guidance and support during the development of this project. The Python with AI course helped me understand how programming concepts can be applied to a practical data-analysis problem.

I am thankful for the learning resources and project guidelines that helped me understand the required features, dataset preparation, analysis process and documentation structure. I also appreciate the opportunity to practice Python, NumPy and Pandas through a project based on student performance data.

Finally, I would like to thank everyone who supported me during the development, testing and documentation of this project.

TABLE OF CONTENTS

1. Project Overview
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Existing System and Proposed System
7. Technologies and Tools
8. System Requirements
9. Dataset Description
10. Data Dictionary
11. Methodology
12. Project Implementation
13. Python Concepts Used
14. NumPy Usage
15. Pandas Usage
16. Functions and Loops
17. Grading and Pass/Fail Logic
18. Analysis and Results
19. Key Features
20. Testing
21. Challenges and Solutions
22. Learning Outcomes
23. Limitations
24. Future Scope
25. Conclusion

1. PROJECT OVERVIEW

Student Performance Analytics System is a Python-based data analysis project designed to process student academic records and generate useful performance information. Instead of manually calculating totals, averages, grades and pass/fail status, the system performs these operations programmatically.

The project follows the assignment requirement to use a student dataset, load it with Pandas, perform numerical calculations using NumPy, apply functions and loops, and produce understandable analytical results. The project uses a CSV file as the data source and can be developed and executed in Jupyter Notebook.

The system analyzes four academic subjects—Python, DBMS, Computer Networks (CN) and Mathematics—along with attendance. It calculates total marks and average marks for every student, assigns grades, determines Pass/Fail status, identifies highest and lowest performers, calculates subject-wise averages and displays top-performing students.

2. INTRODUCTION

Educational institutions collect large amounts of student information such as marks, attendance and department details. When this information is stored in a simple table, it may be difficult to quickly understand overall performance. Data analysis helps convert raw records into useful information for decision-making.

Python is suitable for this type of task because it provides simple programming constructs as well as powerful data-analysis libraries. Pandas provides DataFrame-based data handling, while NumPy supports efficient numerical calculations.

This project applies these technologies to a small student dataset. It is intentionally a basic data-analysis system rather than an advanced machine-learning application, matching the project guidelines.

The final output is designed to be readable and useful for a student, teacher or evaluator. It provides both individual-level and class-level information.

3. PROBLEM STATEMENT

Manual student performance analysis requires repeated calculations. For each student, total marks and average marks must be calculated, a grade must be assigned, and pass/fail status must be checked. Additional calculations are needed to find the highest performer, lowest performer, best subject and overall class average.

Manual analysis becomes more time-consuming as the number of student records increases. It can also introduce calculation errors and make it difficult to compare students consistently.

The proposed system solves this problem by using a structured CSV dataset and Python code. Once the dataset is loaded, the system performs the required calculations automatically and presents the results in a clear form.

The project therefore demonstrates how basic programming and data-analysis skills can be combined to solve a practical academic problem.

4. OBJECTIVES

- Create or use a realistic student dataset containing student details, marks and attendance.
- Load the dataset into a Pandas DataFrame.
- Use NumPy for numerical calculations where appropriate.
- Calculate total marks and average marks for each student.
- Assign grades using clearly defined conditions.
- Determine Pass/Fail status using marks and attendance criteria.
- Calculate overall class performance.
- Perform subject-wise performance analysis.
- Identify highest and lowest performing students.
- Display the top-performing students.
- Use reusable functions and loops in the implementation.
- Organize the project so it can be tested and uploaded to GitHub.

5. SCOPE OF THE PROJECT

The scope of the Student Performance Analytics System is limited to basic academic data analysis. It focuses on structured student records and does not attempt to predict future marks or use advanced machine-learning algorithms.

5.1 Included Scope

- Student record loading from CSV.
- DataFrame inspection and organization.
- Subject-wise marks processing.
- Total and average calculation.
- Grade assignment.
- Pass/Fail classification.
- Attendance-based evaluation.
- Class average calculation.
- Subject-wise average calculation.
- Highest, lowest and top-five analysis.

5.2 Out of Scope

- Advanced machine learning prediction.
- Real-time integration with a college ERP.
- Online student login and authentication.
- Database server deployment.
- Automatic generation of official academic certificates.

6. EXISTING SYSTEM AND PROPOSED SYSTEM

6.1 Existing / Manual Approach

In a manual approach, marks are entered into a spreadsheet or register and calculations are performed separately. The teacher or student may need to calculate totals, averages and grades repeatedly. Comparing all students and subjects may take additional time.

6.2 Proposed Approach

The proposed Python system reads the complete dataset once and performs multiple calculations programmatically. Pandas is used to organize and manipulate the records, NumPy is used for numerical calculations, and Python functions and conditions are used to implement project rules.

Aspect	**Manual Approach**	**Proposed System**
Calculation	Manual/repeated	Automatic through code
Data handling	Register/spreadsheet	Pandas DataFrame
Numerical analysis	Manual formulas	NumPy/Pandas
Grade assignment	Manual checking	Function + conditions
Comparison	Time-consuming	Sorting/filtering
Repeatability	Low	High
Scalability	Limited	Easier to extend

7. TECHNOLOGIES AND TOOLS

Technology / Tool	Purpose
Python	Core programming language, logic, functions and conditions
NumPy	Numerical calculations and statistical operations
Pandas	CSV loading, DataFrame creation, filtering, sorting and analysis
Jupyter Notebook	Interactive development and execution
CSV	Storage format for student records
GitHub	Project version control and submission repository

Python was selected because it is readable and has strong support for data analysis. Pandas is particularly useful for tabular data, while NumPy provides efficient numerical operations.

8. SYSTEM REQUIREMENTS

8.1 Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM recommended
- At least 1 GB free storage for the project and Python environment
- Keyboard and mouse/touchpad

8.2 Software Requirements

- Windows, Linux or macOS
- Python 3.x
- Jupyter Notebook or JupyterLab
- Pandas library
- NumPy library
- Web browser for GitHub and optional Google Colab

8.3 Installation

```
pip install numpy pandas jupyter
```

If Jupyter Notebook is already installed, the project can be opened directly from the folder containing the notebook and student.csv.

9. DATASET DESCRIPTION

The project dataset contains 20 student records, satisfying the assignment guideline to use approximately 15–20 realistic records. Each record contains a unique Student ID, student name, department, marks in four subjects and attendance percentage.

The CSV format is simple, portable and easy to inspect. It can be opened using spreadsheet software as well as loaded directly into Pandas.

Column	Description	Data Type
Student_ID	Unique student identifier	Text
Name	Student name	Text
Department	Academic department	Text
Python	Python subject marks	Numeric
DBMS	DBMS subject marks	Numeric
CN	Computer Networks marks	Numeric
Maths	Mathematics marks	Numeric
Attendance	Attendance percentage	Numeric

10. DATA DICTIONARY AND DATA QUALITY

A data dictionary defines the meaning of each field so that the dataset can be interpreted consistently.

Field	Valid Range / Rule	Purpose
Student_ID	Unique value	Identify student
Name	Non-empty text	Identify student by name
Department	Text category	Group students
Python	0–100	Subject score
DBMS	0–100	Subject score
CN	0–100	Subject score
Maths	0–100	Subject score
Attendance	0–100%	Attendance measure

10.1 Data Quality Checks

- Check number of records using DataFrame shape.
- Check column names using DataFrame columns.
- Check data types using DataFrame info().
- Check for missing values using isnull().sum().
- Check descriptive statistics using describe().
- Ensure marks and attendance are numeric before calculations.

11. METHODOLOGY

The project follows a simple data-analysis pipeline. First, the dataset is prepared in CSV format. Next, Pandas loads the file into a DataFrame. The data is inspected for structure and missing values. After validation, calculations are performed.

The four subject marks are added to calculate total marks. The total is divided by four to calculate average marks. A grading function uses conditional statements to assign a grade. A second rule checks average marks and attendance to determine Pass or Fail.

After individual calculations, the system performs class-level analysis. It calculates the overall class average, subject-wise averages, highest and lowest performers, and top five students.

Finally, the results are displayed in the notebook. Screenshots of these outputs can be added to the documentation before final submission.

11.1 Flow

Dataset → Load with Pandas → Validate → Calculate Total → Calculate Average → Grade → Pass/Fail → Subject Analysis → Ranking → Final Output

12. PROJECT IMPLEMENTATION

12.1 Import Libraries

```
import pandas as pd
import numpy as np
```

Pandas is imported as pd and NumPy as np. These aliases are commonly used in Python data-analysis programs.

12.2 Load Dataset

```
df = pd.read_csv("student.csv")
print(df.head())
```

The read_csv() function reads the CSV file and creates a DataFrame named df.

12.3 Inspect Dataset

```
print(df.shape)
print(df.columns)
print(df.info())
print(df.isnull().sum())
```

These statements help verify the structure and quality of the input data.

13. PYTHON CONCEPTS USED

13.1 Variables and Data Types

Variables are used to store values such as totals, averages, grade results and counts. The project uses strings for names and departments and numeric values for marks and attendance.

13.2 Conditional Statements

if, elif and else statements are used for grade assignment and Pass/Fail classification.

13.3 Functions

Functions make repeated logic reusable. The grade calculation is implemented as a function so the same rule can be applied to every student.

13.4 Loops

A loop can be used to iterate over subject names and calculate subject-wise averages. Iteration also helps demonstrate core Python programming concepts required by the assignment.

13.5 Lists

A list of subject column names is useful when applying the same operation to multiple subjects.

14. NUMPY USAGE

NumPy is used for numerical operations. In this project, NumPy can be used to calculate the class-level mean of average marks and to work with numerical arrays.

The use of NumPy demonstrates that numerical calculations can be performed efficiently on numeric data. NumPy also provides functions such as mean(), max(), min() and array operations.

14.1 Why NumPy?

- Efficient numerical computation.
- Useful statistical functions.
- Works naturally with numerical arrays and Pandas data.
- Helps demonstrate the NumPy requirement of the assignment.

15. PANDAS USAGE

Pandas is the main data-handling library used in the project. It provides the DataFrame structure used to store the complete student dataset.

```
df = pd.read_csv('student.csv')
df['Total_Marks'] = df[['Python','DBMS','CN','Maths']].sum(axis=1)
df['Average_Marks'] = df['Total_Marks'] / 4
```

Pandas is also used to sort students, filter Pass/Fail records, calculate subject averages and select the top five students.

```
top_students = df.sort_values('Average_Marks', ascending=False).head(5)
```

These operations reduce repetitive manual calculations and make the analysis reproducible.

1 6. FUNCTIONS AND LOOPS

16.1 Grade Function

```
def calculate_grade(avg):
    if avg >= 90:
        return "A+"
    elif avg >= 80:
        return "A"
    elif avg >= 70:
        return "B"
    elif avg >= 60:
        return "C"
    elif avg >= 50:
        return "D"
    else:
        return "F"

df["Grade"] = df["Average_Marks"].apply(calculate_grade)
```

16.2 Pass/Fail Function

```
def pass_fail(avg, attendance):
    if avg >= 40 and attendance >= 75:
        return "Pass"
    return "Fail"

df["Status"] = df.apply(
    lambda row: pass_fail(row["Average_Marks"], row["Attendance"]),
    axis=1
)
```

16.3 Subject Loop

```
subjects = ["Python", "DBMS", "CN", "Maths"]
for subject in subjects:
    print(subject, df[subject].mean())
```

17. GRADING AND PASS/FAIL LOGIC

Average Marks	Grade
90–100	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

The grading rule is based on average marks. A student receives A+ for an average of 90 or above, A for 80 or above, B for 70 or above, C for 60 or above, D for 50 or above, and F below 50.

Pass condition: Average Marks ≥ 40 AND Attendance ≥ 75%. If either condition is not satisfied, the student is marked Fail. This rule combines academic performance and attendance.

The rule is implemented consistently for every student through reusable functions. The same logic can be changed later if the institution specifies different grading or attendance criteria.

18. ANALYSIS AND RESULTS

The analysis stage converts the processed DataFrame into meaningful findings. The exact values shown in the final submission should be taken from the student's latest executed Jupyter Notebook output.

18.1 Overall Class Average

```
overall_average = np.mean(df['Average_Marks'])
print(round(overall_average, 2))
```

18.2 Highest Performer

```
highest = df.loc[df['Average_Marks'].idxmax()]
print(highest[['Name','Average_Marks','Grade']])
```

18.3 Lowest Performer

```
lowest = df.loc[df['Average_Marks'].idxmin()]
print(lowest[['Name','Average_Marks','Grade']])
```

18.4 Pass/Fail Statistics

```
print(df['Status'].value_counts())
```

18.5 Subject-wise Average

```
for subject in subjects:
    avg = df[subject].mean()
    print(f"{subject}: {avg:.2f}")
```

18.6 Top Five

```
print(df.sort_values('Average_Marks', ascending=False).head(5))
```

19. KEY FEATURES

- CSV-based student dataset.
- Pandas DataFrame for structured data.
- NumPy-based numerical analysis.
- Automatic total and average calculation.
- Automatic grade assignment.
- Attendance-aware Pass/Fail classification.
- Highest and lowest performer identification.
- Subject-wise performance analysis.
- Top-five ranking.
- Readable notebook output.
- Reusable functions.
- Loop-based subject analysis.
- Easy extension for additional students or subjects.

19.1 Benefits

The system saves calculation time, provides consistent results, reduces repetitive manual work and makes the analysis easy to reproduce. It also provides a practical way to demonstrate basic Python programming and data-analysis concepts.

20. TESTING

Testing is performed to confirm that the notebook executes without avoidable errors and that the calculations follow the defined project rules.

Test Case	Input / Condition	Expected Result
CSV loading	Valid student.csv	DataFrame loads successfully
Record count	20 records	Shape shows 20 student rows
Missing values	No unexpected blanks	Missing-value check is acceptable
Total marks	Four subject marks	Correct row-wise sum
Average	Total / 4	Correct average
Grade	Average in each range	Correct grade
Pass	Average ≥40 and attendance ≥75	Pass
Fail	Any pass condition false	Fail
Highest	Maximum average	Correct student selected
Lowest	Minimum average	Correct student selected
Top 5	Sort descending	Five highest averages displayed

Before final submission, the student should execute every notebook cell from top to bottom and confirm that the output is generated correctly.

21. CHALLENGES AND SOLUTIONS

Challenge	Solution / Learning
Understanding CSV structure	Inspected columns and DataFrame information using Pandas.
Calculating repeated values	Created reusable calculations and used DataFrame operations.
Applying grades consistently	Created a grade function using conditions.
Combining marks and attendance	Implemented a clear Pass/Fail rule.
Subject-wise comparison	Used a subject list and loop.
Finding top performers	Sorted Average_Marks in descending order.
Notebook execution errors	Ran cells in sequence and checked file paths.
Project organization	Kept notebook, CSV and documentation together.

These challenges helped connect classroom Python concepts with a complete practical workflow.

22. LEARNING OUTCOMES

- Understanding how to work with CSV datasets.
- Understanding Pandas DataFrames.
- Using NumPy for numerical calculations.
- Writing functions for reusable logic.
- Using loops for repeated analysis.
- Applying if/elif/else conditions.
- Filtering and sorting DataFrame records.
- Performing basic statistical analysis.
- Interpreting subject-wise and student-wise results.
- Testing a Python notebook systematically.
- Preparing documentation for a software/data-analysis project.
- Understanding basic GitHub project submission workflow.

The project also improves problem-solving ability because the developer must translate a real-world requirement into a sequence of data-processing steps.

23. LIMITATIONS

- The project uses a small sample dataset.
- The data is static and stored in CSV format.
- Results depend on the quality and correctness of input data.
- The system does not provide predictive analytics.
- There is no web-based user interface.
- There is no database backend.
- Attendance and marks are assumed to be correctly entered.
- The grading criteria are fixed in the program.

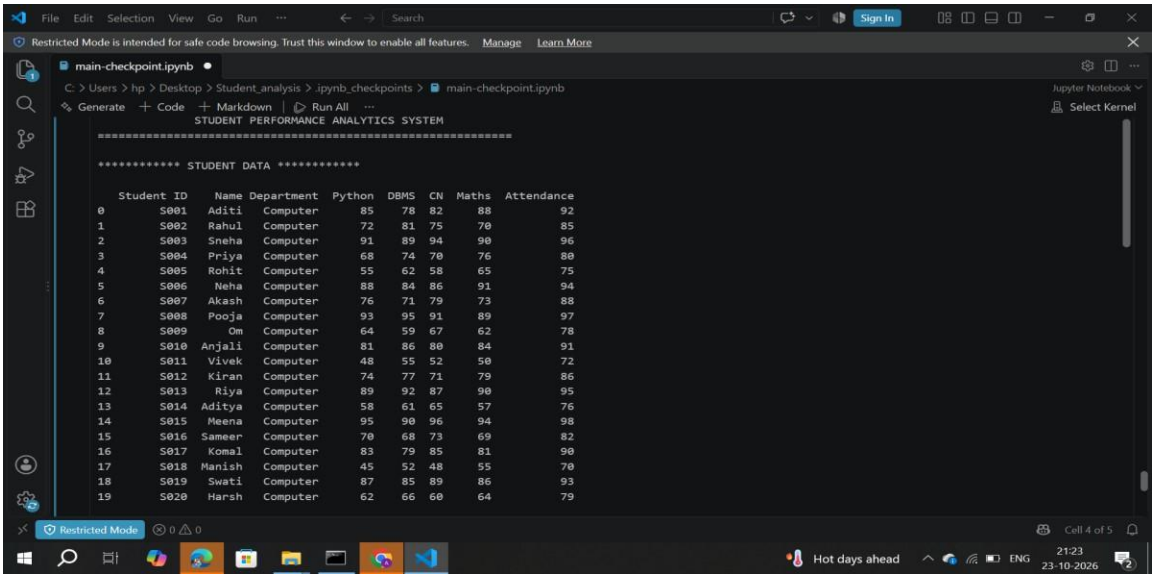
These limitations are acceptable for a beginner-level Python data-analysis assignment. They also provide opportunities for future development.

24. FUTURE SCOPE

- Add a larger student dataset.
- Connect the system to a SQL database.
- Build a web interface using Flask or Streamlit.
- Add charts for subject-wise performance.
- Generate downloadable PDF or Excel reports.
- Add student search and filtering.
- Add semester-wise performance tracking.
- Add attendance trend analysis.
- Add predictive machine-learning models in a future advanced version.
- Create an administrator dashboard.
- Add secure login and role-based access.

The current system can therefore act as a foundation for a larger academic analytics application.

25. OUTPUT

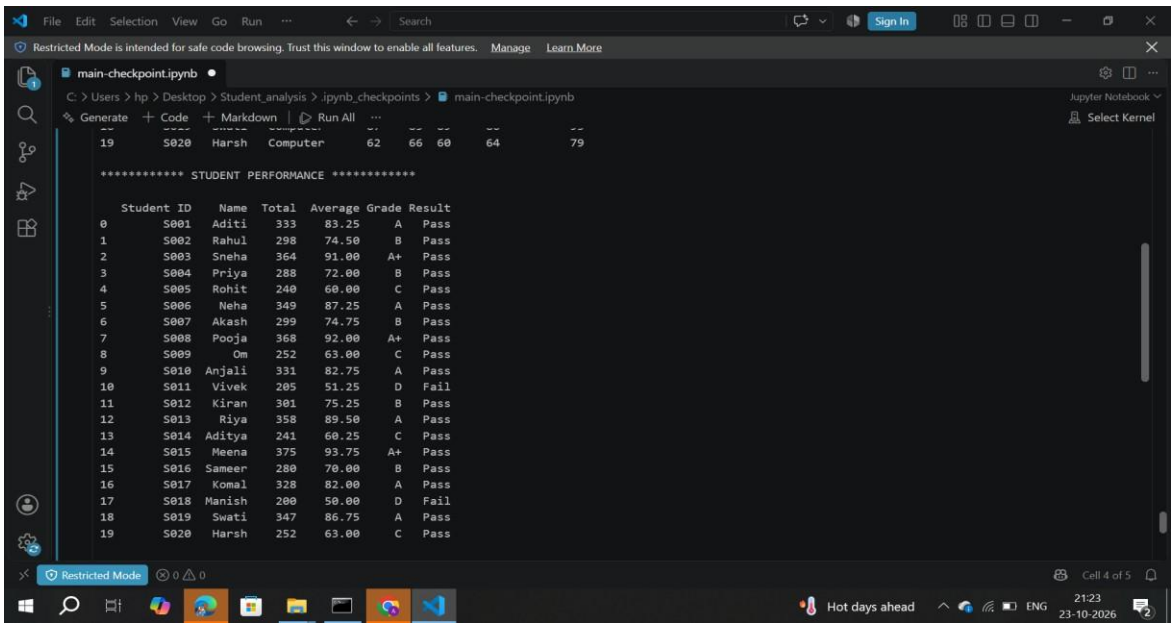


```
STUDENT PERFORMANCE ANALYTICS SYSTEM
============================================================

************ STUDENT DATA ************

   Student ID     Name Department  Python  DBMS  CN  Maths  Attendance
0        S001   Aditi   Computer      85    78  82     88          92
1        S002   Rahul   Computer      72    81  75     70          85
2        S003   Sneha   Computer      91    89  94     90          96
3        S004   Priya   Computer      68    74  70     76          80
4        S005   Rohit   Computer      55    62  58     65          75
5        S006    Neha   Computer      88    84  86     91          94
6        S007   Akash   Computer      76    71  79     73          88
7        S008   Pooja   Computer      93    95  91     89          97
8        S009      Om   Computer      64    59  67     62          78
9        S010  Anjali   Computer      81    86  80     84          91
10       S011   Vivek   Computer      48    55  52     50          72
11       S012   Kiran   Computer      74    77  71     79          86
12       S013    Riya   Computer      89    92  87     90          95
13       S014  Aditya   Computer      58    61  65     57          76
14       S015   Meena   Computer      95    90  96     94          98
15       S016  Sameer   Computer      70    68  73     69          82
16       S017   Komal   Computer      83    79  85     81          90
17       S018  Manish   Computer      45    52  48     55          70
18       S019   Swati   Computer      87    85  89     86          93
19       S020   Harsh   Computer      62    66  60     64          79
```

Fig- Student Data



```
************ STUDENT PERFORMANCE ************

   Student ID     Name  Total  Average Grade Result
0        S001   Aditi    333    83.25     A   Pass
1        S002   Rahul    298    74.50     B   Pass
2        S003   Sneha    364    91.00    A+   Pass
3        S004   Priya    288    72.00     B   Pass
4        S005   Rohit    240    60.00     C   Pass
5        S006    Neha    349    87.25     A   Pass
6        S007   Akash    299    74.75     B   Pass
7        S008   Pooja    368    92.00    A+   Pass
8        S009      Om    252    63.00     C   Pass
9        S010  Anjali    331    82.75     A   Pass
10       S011   Vivek    205    51.25     D   Fail
11       S012   Kiran    301    75.25     B   Pass
12       S013    Riya    358    89.50     A   Pass
13       S014  Aditya    241    60.25     C   Pass
14       S015   Meena    375    93.75    A+   Pass
15       S016  Sameer    280    70.00     B   Pass
16       S017   Komal    328    82.00     A   Pass
17       S018  Manish    200    50.00     D   Fail
18       S019   Swati    347    86.75     A   Pass
19       S020   Harsh    252    63.00     C   Pass
```

Fig-Student Performance

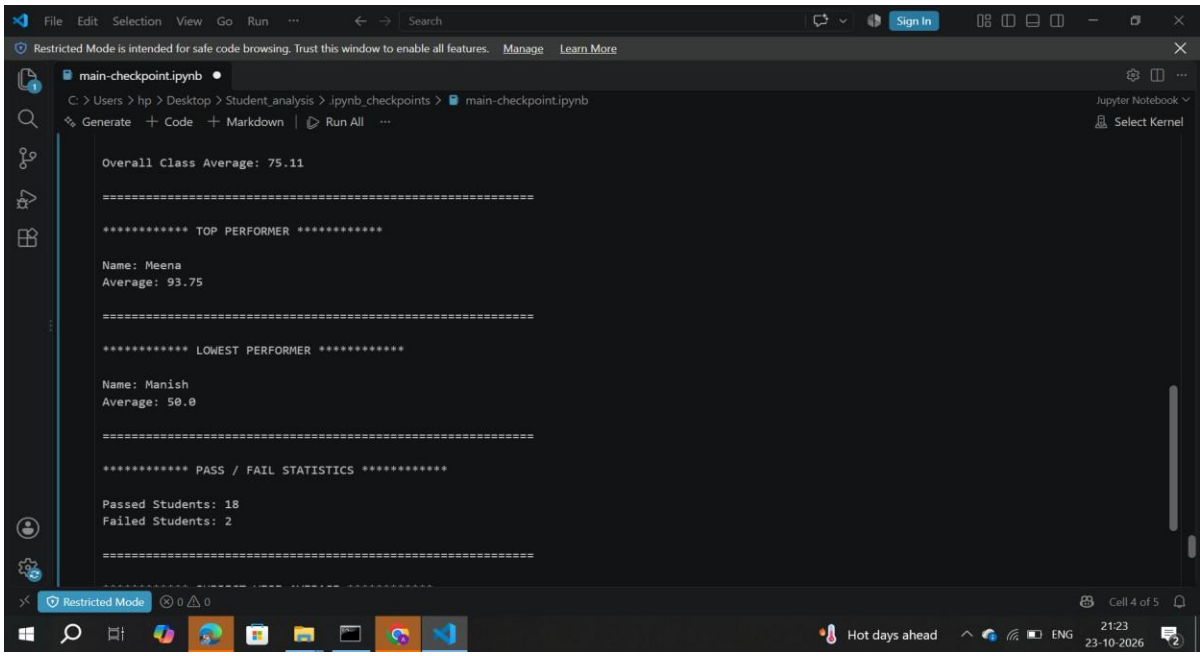
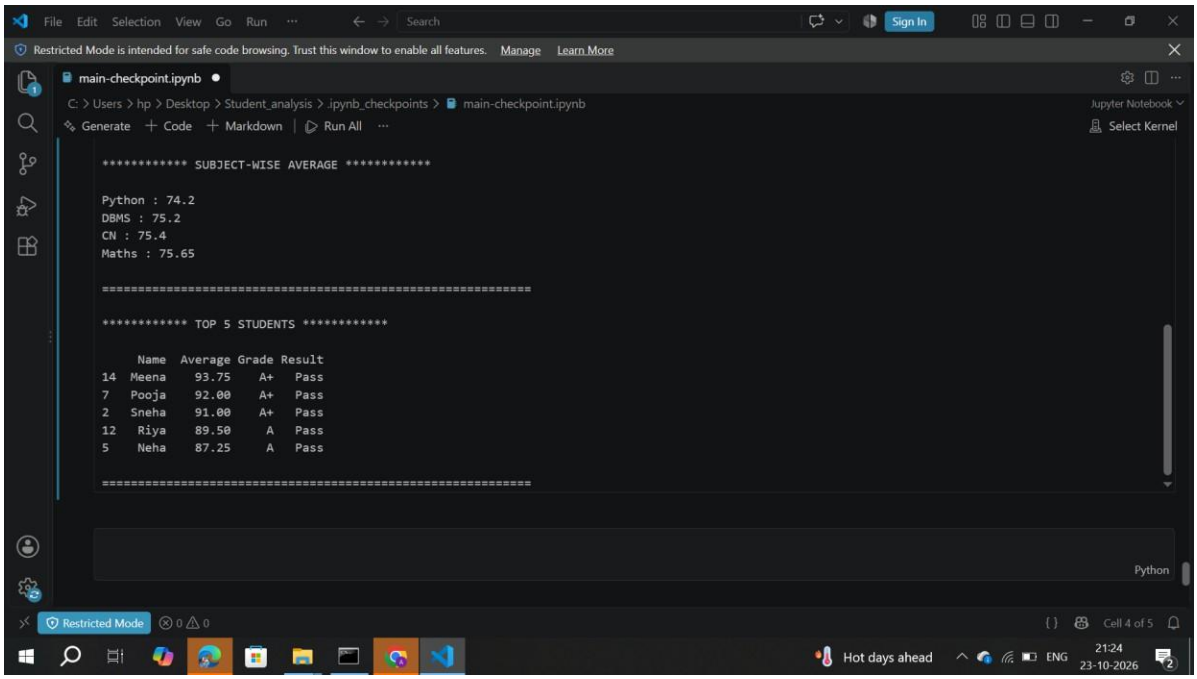


Fig- Student Performance Analysis



26. CONCLUSION

The Student Performance Analytics System successfully demonstrates how Python can be used to analyze structured academic data. The project loads student records from a CSV file, organizes them using Pandas and performs numerical calculations using NumPy.

The system calculates total marks, average marks, grades and Pass/Fail status. It also provides class-level and subject-level analysis and identifies top-performing students. Functions, loops and conditional statements make the implementation reusable and understandable.

The project meets the main academic goals of a Python data-analysis assignment and provides practical experience in dataset handling, programming logic, testing and documentation.

27. REFERENCES

- EWB Courses – Python with AI Project Guidelines provided for the assignment.
- Python documentation and classroom learning materials.
- NumPy concepts covered during the Python with AI course.
- Pandas concepts covered during the Python with AI course.
- Jupyter Notebook environment used for interactive development.
- GitHub repository used for project submission and version control.